Computer Vision DermaCheck: clasificación de lesiones cutáneas con MobileNetV2 en una Raspberry Pi 5

Salvador Martinez[1*], Christopher Yañez[1*], Mauricio Chavez[1†] and Miguel Valdez[1†]

[1*]PrepaTec Campus Cuernavaca, Tecnológico de Monterrey, Autopista del Sol km 104+060, Col. Real del Puente, Xochitepec, 62790, Morelos, México.

*Autores de correspondencia. Correos: a01425829@tec.mx; a01426168@tec.mx;
Coautores: a01426058@tec.mx; a01739293@tec.mx;
†Estos autores contribuyeron por igual a este trabajo.

Resumen

Distinguir a simple vista un melanoma de un lunar común es difícil incluso para personal entrenado, y el dermatoscopio que facilita esa tarea no suele estar al alcance fuera de un consultorio. En este trabajo armamos Computer Vision DermaCheck, un prototipo de bajo costo que clasifica fotografías de lesiones cutáneas en las ocho categorías del conjunto ISIC 2019. Partimos de una red MobileNetV2 preentrenada con ImageNet y la ajustamos en dos fases con 18,237 imágenes dermatoscópicas, ponderando la pérdida para compensar el fuerte desbalance entre clases; todo el entrenamiento se hizo en el CPU de una laptop y tomó 44.6 minutos. El modelo, exportado a TensorFlow Lite, corre en una Raspberry Pi 5 dentro de una aplicación táctil que muestra la imagen en vivo, un cuadro guía y las tres clases más probables. Sobre 5,067 imágenes que la red nunca vio, obtuvo una exactitud de 0.725 y un recall promedio por clase de 0.632; reconoce bien los nevos (0.855) y las lesiones vasculares (0.862), pero solo el 47.1 % de los melanomas. En la Pi, cada imagen tarda 9.7 ms con cuatro hilos y, con el modelo completo, las predicciones coinciden con las de la laptop en los 5,067 casos. Las pruebas de carga mostraron que la alimentación y la temperatura condicionan el uso real de la placa tanto como el propio modelo. El sistema tiene fines educativos y no sustituye un diagnóstico médico.

Palabras clave: lesiones cutáneas, melanoma, dermatoscopia, aprendizaje por transferencia, MobileNetV2, cómputo en el borde, Raspberry Pi

1. Introducción

El melanoma ocupa un lugar modesto en las tablas de incidencia, pero pesa mucho en las de mortalidad. Según las estimaciones de GLOBOCAN para 2024, en el mundo se registraron 337,969 casos nuevos de melanoma de piel y 58,398 muertes por esa causa [1]; en México, la misma fuente calcula 2,461 casos nuevos y 809 defunciones en un año [2]. Reconocerlo a tiempo importa, y ahí aparece el primer obstáculo: a simple vista, la exactitud clínica para reconocer un melanoma ronda el 60 % [3].

La dermatoscopia, que observa la lesión con aumento y luz polarizada, mejora ese panorama. Un metaanálisis de 27 estudios encontró que, en manos de examinadores con experiencia, eleva significativamente la capacidad diagnóstica frente a la inspección sin aparato (log odds ratio de 4.0 contra 2.7) [3]. El problema se desplaza entonces hacia el acceso, porque el equipo y el entrenamiento para interpretarlo se concentran en consultorios especializados.

Las redes neuronales convolucionales abrieron otra ruta. Esteva y colaboradores entrenaron una red con 129,450 imágenes clínicas y la compararon con 21 dermatólogos certificados en tareas de clasificación binaria; el desempeño de la red quedó a la par del de los especialistas [4]. Poco después, Haenssle y su equipo enfrentaron una red entrenada con imágenes dermatoscópicas contra 58 dermatólogos: con la misma sensibilidad, la red alcanzó una especificidad de 82.5 %, mayor que la de los médicos [5]. Estos sistemas, sin embargo, suelen vivir en servidores con tarjetas gráficas y no siempre se sostienen fuera de sus datos de origen; con fotografías de pieles oscuras, por ejemplo, su rendimiento cae de forma marcada [6].

Nuestro proyecto explora el extremo opuesto: qué tanto se logra con hardware barato y con herramientas abiertas. Construimos un clasificador de ocho tipos de lesión basado en MobileNetV2 [7], una arquitectura pensada para dispositivos móviles, lo entrenamos con el conjunto ISIC 2019 [8] en el CPU de una laptop y lo desplegamos en una Raspberry Pi 5 con pantalla táctil. Además del modelo, documentamos dos aspectos prácticos: cuánto cambia el resultado al pasar de una computadora a otra y qué exige la placa en energía y temperatura para funcionar sin interrupciones. El sistema es una herramienta educativa; no está validado clínicamente ni pretende diagnosticar.

2. Resultados

La Fig. 1 resume el recorrido completo. Las imágenes se preparan y la red se entrena en la laptop; el archivo resultante viaja por el repositorio del proyecto hasta la Raspberry Pi, donde una aplicación lo usa para clasificar lo que capta la cámara. Los resultados de esta sección provienen de las bitácoras de entrenamiento, de la evaluación sobre el conjunto de prueba y de mediciones hechas directamente en la placa.

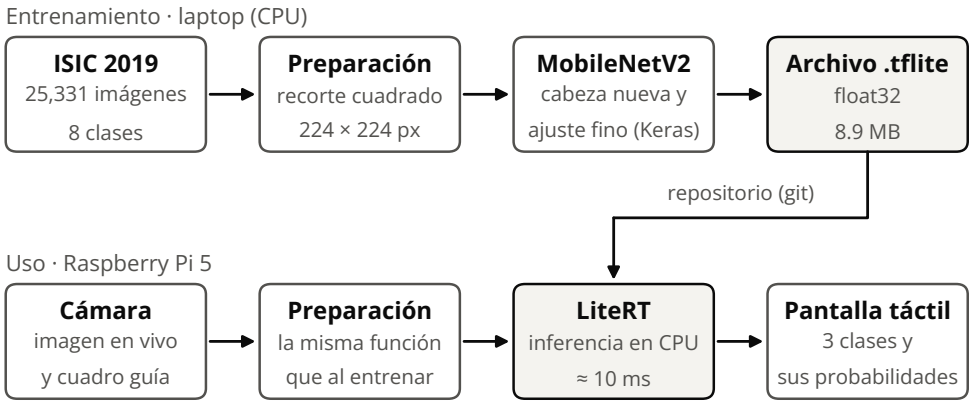


Fig. 1 Arquitectura de Computer Vision DermaCheck. Arriba, las etapas que corren una sola vez en la laptop; abajo, el uso diario en la Raspberry Pi 5. La función de preparación de la imagen es idéntica en ambos lados, de modo que la red recibe en la placa exactamente el mismo tipo de entrada con el que aprendió.

2.1. Entrenamiento

El entrenamiento completo duró 44.6 minutos en el CPU de una laptop con procesador Apple M4, sin aceleración gráfica. La primera fase, en la que solo aprende la capa final, avanzó despacio: tras seis épocas el recall promedio de validación apenas llegó a 0.441 (Fig. 2). El salto vino al descongelar las últimas 40 capas de MobileNetV2. Tras un tropiezo en la primera época de esa fase, la curva subió con rapidez, y cuando el paso de aprendizaje se redujo a la mitad, en la época 17, el recall ganó más de nueve centésimas de golpe. El máximo, 0.670, apareció en la época 19, y ese es el modelo que conservamos.

La pérdida de validación se estabilizó cerca de 0.71 mientras la de entrenamiento seguía bajando hasta 0.53. Esa brecha creciente indica que la red empezaba a memorizar detalles del conjunto de entrenamiento, y es otra razón para no alargar la segunda fase sin más datos o más aumento de imágenes.

2.2. Desempeño en el conjunto de prueba

Evaluamos el modelo una sola vez sobre 5,067 imágenes reservadas desde el inicio, que no intervinieron ni en el ajuste de los pesos ni en la elección de la mejor época. La exactitud global fue de 0.725 (3,672 aciertos) y el recall promedio por clase, de 0.632, algo por debajo del 0.670 de validación. La Tabla 1 desglosa el resultado por categoría.

Las cifras más altas corresponden a los nevos (0.855) y a las lesiones vasculares (0.862), cuyo aspecto rojizo y bien delimitado las vuelve fáciles de separar. En el otro extremo quedan el carcinoma escamoso (0.419), la queratosis actínica (0.451) y el melanoma (0.471). La matriz de confusión de la Fig. 3 explica buena parte de esos tropiezos: de los 949 melanomas, 298 (31.4 %) fueron tomados por nevos y 111 (11.7 %) por queratosis benignas, y un tercio de los carcinomas escamosos se confundió

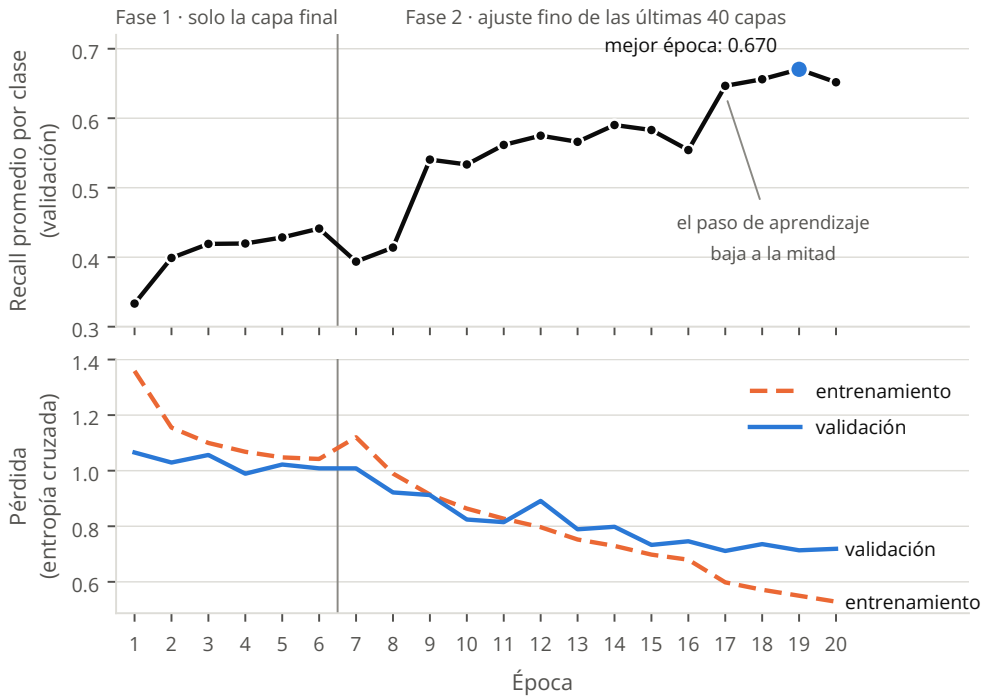


Fig. 2 Evolución del entrenamiento. Arriba, el recall promedio por clase en el conjunto de validación, que es la métrica que decidió qué pesos conservar; abajo, la pérdida en entrenamiento y en validación. La línea vertical separa la fase en la que solo aprende la capa final de la fase de ajuste fino.

con carcinomas basocelulares. El dermatofibroma muestra la combinación contraria, buen recall con precisión muy baja (0.234): el modelo lo propone con frecuencia para lesiones que no lo son, efecto esperable del peso alto que recibió esa clase tan escasa.

Si agrupamos las tres clases malignas (melanoma, carcinoma basocelular y carcinoma escamoso) contra el resto, el modelo identifica como maligna alguna de esas tres en 1,102 de 1,672 lesiones malignas, una sensibilidad de 0.659, y descarta correctamente 3,072 de 3,395 lesiones no malignas, una especificidad de 0.905.

2.3. Modelo completo y modelo ligero

Exportamos dos versiones del modelo. La primera conserva los pesos en punto flotante de 32 bits; la segunda los cuantiza a enteros de 8 bits, lo que reduce el archivo de 8.9 a 2.5 MB. La Tabla 2 compara ambas. La versión float32 reprodujo las probabilidades del modelo original con una diferencia máxima de 9.2×10^{-6} en las 5,067 imágenes, así que para fines prácticos es el mismo modelo. La ligera, en cambio, eligió la misma clase que el original en el 94.5 % de las imágenes y perdió entre uno y 1.4 puntos de recall promedio.

Un detalle nos sorprendió. Con la versión float32, las probabilidades de la Pi y las de la laptop difirieron en menos de tres millonésimas. Con la ligera no: la laptop

Tabla 1 Desempeño por clase en el conjunto de prueba (modelo float32). El recall indica qué fracción de las lesiones de cada clase reconoció el modelo; la precisión, qué fracción de sus predicciones de esa clase fue correcta.

Clase	Precisión	Recall	F1	Imágenes
AK, queratosis actínica	0.425	0.451	0.437	182
BCC, carcinoma basocelular	0.665	0.792	0.723	606
BKL, queratosis benigna	0.483	0.615	0.541	507
DF, dermatofibroma	0.234	0.595	0.336	42
MEL, melanoma	0.761	0.471	0.582	949
NV, nevo melanocítico	0.850	0.855	0.852	2,606
SCC, carcinoma escamoso	0.422	0.419	0.421	117
VASC, lesión vascular	0.667	0.862	0.752	58
Promedio por clase	0.563	0.632	0.580	5,067
Promedio ponderado	0.742	0.725	0.725	5,067

La exactitud global es 0.725. Los promedios ponderados usan como peso el número de imágenes de cada clase, por lo que reflejan sobre todo el comportamiento con los nevos, que son la mitad del conjunto.

Tabla 2 Comparación de las dos versiones exportadas. Las métricas son sobre las 5,067 imágenes de prueba; los tiempos de la Raspberry Pi 5 se midieron clasificando una imagen a la vez.

	float32	Ligero (int8)
Tamaño del archivo	8.9 MB	2.5 MB
Exactitud	0.725	0.721–0.722[1]
Recall promedio por clase	0.632	0.619–0.623[1]
Misma clase que el modelo de Keras	100 %	94.5 %
Tiempo por imagen en la Pi, 1 hilo	21.7 ms	19.0 ms
Tiempo por imagen en la Pi, 4 hilos	9.7 ms	11.5 ms

Con 2 hilos, la versión float32 tardó 12.7 ms por imagen en la Pi.

[1] El primer valor se obtuvo en la laptop que hizo la exportación y el segundo en la Raspberry Pi; la diferencia se discute en el texto.

que hizo la exportación obtuvo un recall promedio de 0.619 y la Pi, de 0.623, con diferencias de hasta 0.015 en las probabilidades de las imágenes de muestra. Todo apunta a las rutinas de cálculo entero que el intérprete elige para cada procesador: la misma laptop, con una versión anterior del intérprete, reproduce exactamente los resultados de la Pi. Como la versión ligera además fue más lenta con cuatro hilos, la aplicación usa la float32.

2.4. Funcionamiento en la Raspberry Pi 5

Clasificamos en la propia Pi las 5,067 imágenes de prueba, una por una, como lo hace la aplicación. Con la versión float32, la clase elegida coincidió con la de la laptop en todos los casos. Con los cuatro núcleos, cada imagen tardó 9.7 ms, lo que daría para

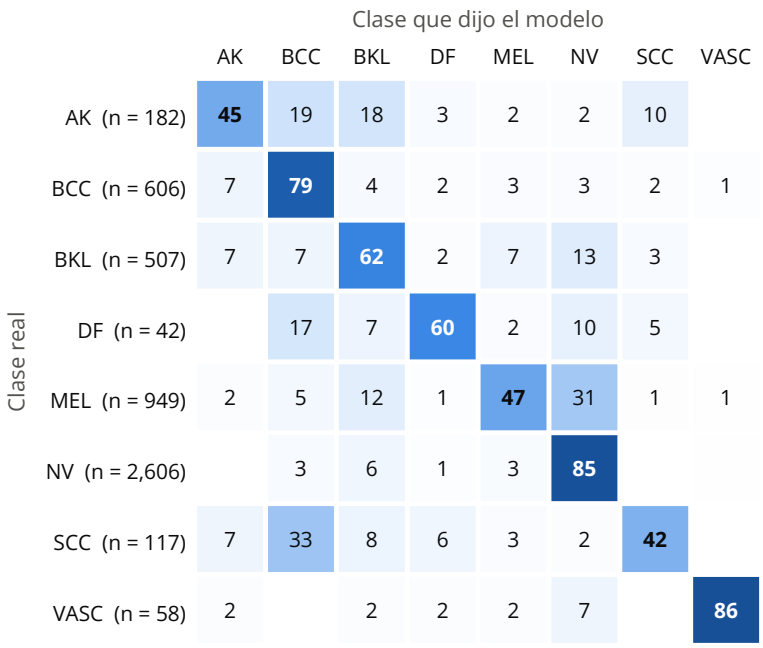


Fig. 3 Matriz de confusión en el conjunto de prueba, normalizada por filas: cada celda indica qué porcentaje de las lesiones de una clase real recibió cada etiqueta. La diagonal corresponde a los aciertos y n es el número de imágenes de cada clase. Se omiten los valores menores a 0.5 %.

clasificar unas cien imágenes por segundo; en la práctica la aplicación solo clasifica cuando el usuario toca el botón, así que el tiempo de respuesta lo domina la interfaz y no la red.

La parte complicada resultó ser la energía. Durante las primeras pruebas la placa se apagó dos veces sin aviso, así que repetimos la clasificación completa registrando cada dos segundos la temperatura, el voltaje de la línea de 5 V y las alertas del firmware. La Tabla 3 resume lo que encontramos.

Tabla 3 Pruebas de carga en la Raspberry Pi 5 sin ventilador, clasificando continuamente el conjunto de prueba. Ambas fuentes negociaron 3 A a 5 V; la fuente oficial de la placa entrega 5 A.

Alimentación	Hilos	Temp. máx.	5 V mín.	Resultado
USB-C de laptop	1	67.5 °C	4.81 V	Estable
USB-C de laptop	2	84.5 °C	4.78 V	Estable, con freno térmico
USB-C de laptop	4	–[1]	4.70 V	Se apagó a los 2 s
Cargador de 65 W	4	87.3 °C	4.76 V	Estable, 7 avisos de voltaje

[1]La placa se apagó antes de que la temperatura subiera; la última lectura fue de 75.2 °C.

Conectada al puerto USB-C de una laptop, la Pi soportó uno y dos núcleos, pero al arrancar los cuatro el voltaje cayó a 4.70 V, el sistema registró una alerta de bajo

voltaje y dos segundos después la placa quedó apagada. Con un cargador de pared de 65 W, que también negocia solo 3 A a 5 V, resistió unos cuatro minutos de carga casi continua, aunque con siete alertas breves de bajo voltaje. La temperatura fue el segundo límite: sin disipador, la placa llegó a 87.3 °C, redujo su frecuencia hasta 1.5 GHz durante cerca de 48 segundos y el tiempo por imagen subió de 9.7 a 11.9 ms. Sumando los rieles internos que reporta el controlador de energía, la placa consumió unos 2 W en reposo y alrededor de 7 W con los cuatro núcleos ocupados; la documentación del fabricante recomienda para este modelo una fuente de 5 A [9, 10].

2.5. La aplicación

La Fig. 4 muestra las dos pantallas de la aplicación en el tamaño de la pantalla táctil de siete pulgadas que usaremos (1024×600 píxeles). En la primera, la imagen en vivo aparece oscurecida salvo un cuadro central con una retícula; ese cuadro es exactamente la región que se clasifica. Al tocar «Tomar», la segunda pantalla presenta la clase más probable con su porcentaje, una escala de 0 a 100 con la posición de las tres primeras clases y un aviso permanente de que el resultado no es un diagnóstico. Si alguna de las clases de riesgo (melanoma, carcinomas o queratosis actínica) alcanza 25 % entre las tres primeras, aparece en color ámbar la sugerencia de acudir a un dermatólogo; si la clase principal no llega a 50 %, la aplicación advierte que el resultado es poco seguro.

Para demostrar el sistema sin cámara, la aplicación incluye un modo de prueba que recorre 62 fotografías con licencia abierta: 40 dermatoscópicas del propio ISIC, cinco por clase y elegidas al azar, y 22 clínicas de Wikimedia Commons, tomadas con cámaras comunes. El modelo acertó la clase de 24 de las 40 imágenes dermatoscópicas y solo de 7 de las 22 clínicas. Son muestras pequeñas, pero la caída coincide con lo que esperábamos de un modelo que aprendió únicamente con imágenes de dermatoscopio.

3. Métodos

3.1. Conjunto de datos

Usamos el conjunto de entrenamiento del reto ISIC 2019, que reúne 25,331 imágenes dermatoscópicas de ocho categorías diagnósticas [8, 11]. Las imágenes provienen de tres colecciones: HAM10000, con 10,015 imágenes [12]; BCN20000, del Hospital Clínic de Barcelona [13]; y las del reto ISIC 2017 [14]. Trabajamos con una copia publicada en Hugging Face que ya divide las imágenes en 20,264 para entrenar y 5,067 para probar [15]; ambas partes suman exactamente las 25,331 imágenes del conjunto de entrenamiento oficial, de modo que nuestra prueba es una porción reservada de ese conjunto y no la prueba oficial del reto. De las 20,264 imágenes separamos de forma estratificada el 10 % para validación (2,027) y dejamos 18,237 para ajustar los pesos. La Tabla 4 muestra el reparto, y la Fig. 5, un ejemplo de cada clase.

3.2. Preparación de las imágenes

Cada imagen pasa por la misma función antes de entrar a la red: se corrige su orientación según los metadatos EXIF, se convierte a RGB, se recorta el cuadrado

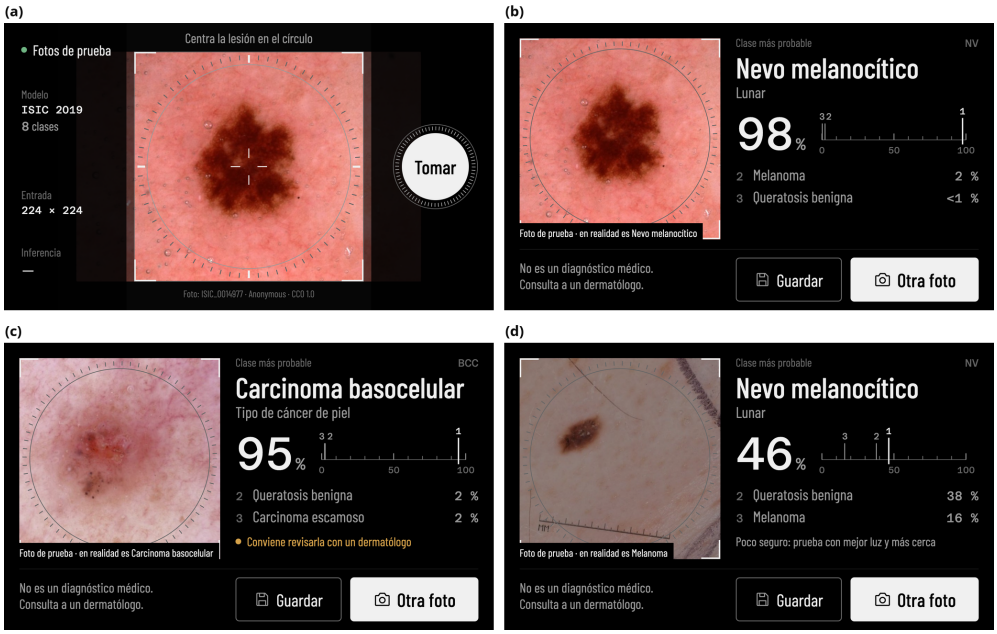


Fig. 4 Capturas de la aplicación con imágenes de prueba de ISIC. (a) Pantalla de cámara con el cuadro guía, que coincide con la región que se clasifica. (b) Nevo reconocido con 98 %. (c) Carcinoma basocelular reconocido con 95 %; el aviso ámbar sugiere consultar a un dermatólogo. (d) Un melanoma que el modelo confunde con un nevo, con solo 46 % y la advertencia de resultado poco seguro.

central más grande posible y se reduce a 224×224 píxeles con interpolación bilineal. La Raspberry Pi usa una copia de esa función, y una prueba automática verifica que ambas producen exactamente los mismos píxeles. La red recibe valores entre 0 y 255; la conversión al intervalo que espera MobileNetV2 ocurre dentro del propio modelo,

$$\tilde{x} = \frac{x}{127.5} - 1, \tag{1}$$

de modo que el programa de la placa no tiene que repetir ese paso. Durante el entrenamiento añadimos variaciones aleatorias a cada imagen: giros horizontales y verticales, rotaciones de hasta media vuelta en cualquier sentido, cambios de escala de hasta 15 % y variaciones de brillo y contraste de la misma magnitud. Estas transformaciones solo existen al entrenar y no forman parte del modelo exportado.

3.3. Arquitectura y entrenamiento

MobileNetV2 se construye con bloques residuales invertidos: cada bloque expande una representación compacta, la filtra con convoluciones separables en profundidad y la vuelve a comprimir con una proyección lineal [7, 16]. Cargamos la red sin su clasificador original, con los pesos aprendidos en ImageNet [17], y le añadimos un promedio global, una capa de *dropout* con tasa 0.3 y una capa densa de ocho salidas

Tabla 4 Número de imágenes por clase en cada partición.

Clase	Entrenamiento	Validación	Prueba	Total
AK, queratosis actínica	616	69	182	867
BCC, carcinoma basocelular	2,445	272	606	3,323
BKL, queratosis benigna	1,905	212	507	2,624
DF, dermatofibroma	177	20	42	239
MEL, melanoma	3,216	357	949	4,522
NV, nevo melanocítico	9,242	1,027	2,606	12,875
SCC, carcinoma escamoso	460	51	117	628
VASC, lesión vascular	176	19	58	253
Total	18,237	2,027	5,067	25,331

La categoría BKL agrupa lentigo solar, queratosis seborreica y queratosis liquenoide. El desbalance es extremo: en la partición de entrenamiento hay 52 nevos por cada dermatofibroma.

con activación softmax. El modelo completo tiene 2,268,232 parámetros. Reutilizar los pesos de ImageNet aprovecha que las primeras capas de una red entrenada con imágenes naturales aprenden rasgos generales, como bordes y manchas de color, que sirven para otras tareas [18].

El entrenamiento siguió dos fases con el optimizador Adam [19] y lotes de 64 imágenes. En la primera, la red base quedó congelada y solo aprendieron los 10,248 parámetros de la capa final, con un paso de aprendizaje de 10^{-3} durante seis épocas. En la segunda descongelamos las últimas 40 capas de MobileNetV2, salvo las de normalización por lotes [20], que se mantuvieron fijas para conservar las estadísticas aprendidas en ImageNet; así quedaron 1,673,608 parámetros entrenables, ajustados con un paso de 10^{-4} durante un máximo de 14 épocas. Usamos TensorFlow 2.21 y Keras 3.15 [21, 22].

Para compensar el desbalance [23], cada imagen de la clase c contribuye a la pérdida con un peso

$$w_c = \left(\frac{N}{K\, n_c} \right)^{1/2}, \tag{2}$$

donde N es el número de imágenes de entrenamiento, $K = 8$ el número de clases y n_c las imágenes de la clase c. La raíz cuadrada suaviza el efecto: el peso va de 0.50 para los nevos a 3.60 para las lesiones vasculares, en lugar del intervalo de 0.25 a 12.95 que daría la proporción inversa directa. La función que se minimiza es la entropía cruzada ponderada sobre cada lote de B imágenes,

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} w_{y_i} \, \log p_{i,y_i}, \tag{3}$$

con y_i la clase verdadera de la imagen i y p_{i,y_i} la probabilidad que el modelo le asigna. La Tabla 5 reúne los hiperparámetros.

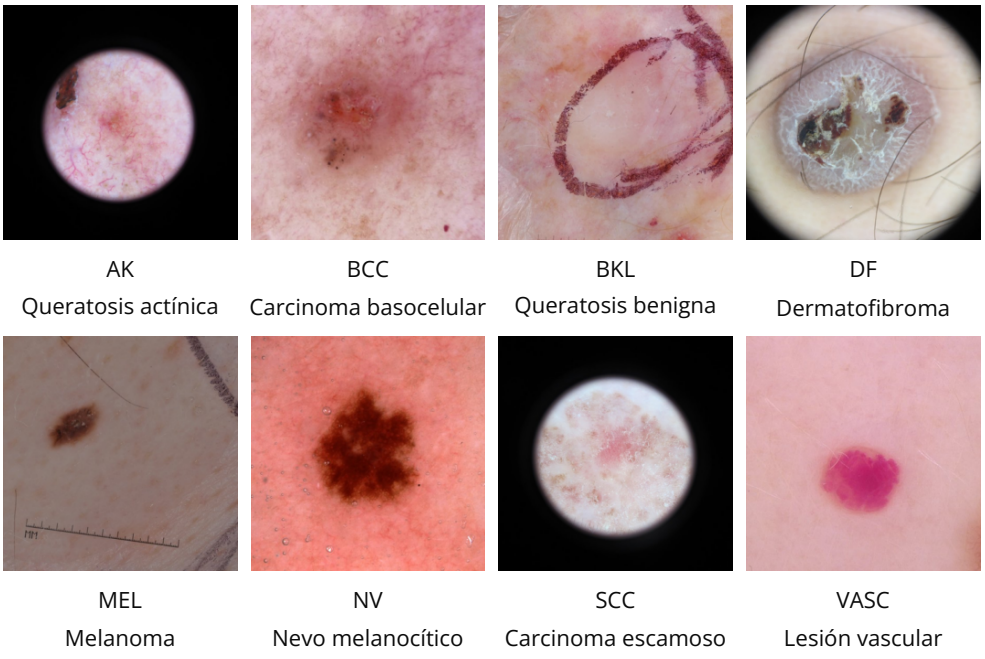


Fig. 5 Una imagen de prueba de cada clase, con el mismo recorte cuadrado que recibe la red. Los créditos y licencias de cada imagen están en el Apéndice A.

Tabla 5 Hiperparámetros del entrenamiento.

Parámetro	Fase 1	Fase 2
Capas entrenables	solo la capa final	últimas 40 de la base y la capa final
Paso de aprendizaje inicial	10^{-3}	10^{-4}
Épocas máximas	6	14
Parámetros entrenables	10,248	1,673,608
Tamaño de lote	64	
Optimizador	Adam	
Dropout	0.3	
Parada temprana	4 épocas sin mejora del recall promedio	
Reducción del paso	a la mitad tras 2 épocas sin mejora	

3.4. Selección del modelo y métricas

La exactitud global engaña cuando la mitad de las imágenes son nevos, porque un modelo que dijera «nevo» siempre ya acertaría la mitad de las veces. Por eso elegimos

los pesos con el recall promedio por clase en validación,

$$\bar{R} = \frac{1}{K} \sum_{c=1}^{K} \frac{\mathrm{VP}_c}{\mathrm{VP}_c + \mathrm{FN}_c}, \tag{4}$$

donde VP_c son las imágenes de la clase c clasificadas correctamente y FN_c las que recibieron otra etiqueta. Al terminar cada época calculamos $\bar{R}$; si mejoraba, guardábamos los pesos. Esa misma métrica controló la parada temprana y la reducción del paso de aprendizaje. En la prueba reportamos además la precisión, $\mathrm{VP}_c/(\mathrm{VP}_c + \mathrm{FP}_c)$, y la puntuación F1, que es la media armónica de precisión y recall.

3.5. Exportación a TensorFlow Lite

El modelo se convirtió al formato de TensorFlow Lite, que Google renombró LiteRT en 2024 [24, 25]. La versión ligera usa cuantización de rango dinámico: los pesos se guardan como enteros de 8 bits y se traducen a reales mediante la relación afín

$$r = S\,(q - Z), \tag{5}$$

en la que q es el entero almacenado, S una escala y Z el entero que representa al cero [26]. Durante la inferencia, el intérprete también cuantiza al vuelo las activaciones, y ese paso explica que el resultado varíe ligeramente entre procesadores. Junto con cada modelo exportamos 16 imágenes de prueba con las probabilidades esperadas; al instalar el modelo, la Pi las clasifica y compara sus resultados con los de la laptop, con una tolerancia de 10^{-3} para la versión float32 y de 0.05 para la ligera.

3.6. Hardware y aplicación

El dispositivo es una Raspberry Pi 5 con 8 GB de memoria, cuyo procesador Broadcom BCM2712 tiene cuatro núcleos Arm Cortex-A76 a 2.4 GHz [27], con Raspberry Pi OS de 64 bits, Python 3.13 y el intérprete `ai-edge-litert` [28]. La pantalla es táctil, de siete pulgadas y 1024×600 píxeles, conectada por HDMI. La aplicación consta de un pequeño servidor en Python que lee la cámara y ejecuta el modelo, y de una página que Chromium muestra a pantalla completa; acepta el módulo de cámara de la Raspberry Pi, una cámara web USB o, sin cámara, las fotografías del modo de prueba. Las imágenes solo se guardan si el usuario lo pide. El Algoritmo 1 describe lo que ocurre al tocar «Tomar».

Para medir el rendimiento en la placa clasificamos las 5,067 imágenes de prueba con uno, dos y cuatro hilos, igual que en la aplicación. Durante esas corridas, un proceso aparte registró cada dos segundos la temperatura del procesador, las alertas de reducción de frecuencia y de bajo voltaje del firmware y el voltaje de la línea de 5 V que mide el controlador de energía, y guardó los mensajes del sistema en la tarjeta de memoria para conservarlos aunque la placa se apagara.

Declaración ética. El estudio no involucró personas ni muestras humanas. Todas las imágenes provienen de colecciones públicas y anonimizadas, con licencias que permiten su uso académico, y se citan con su crédito correspondiente.

Algoritmo 1 Clasificación al tocar «Tomar»

Entrada: cuadro más reciente de la cámara I de $w \times h$ píxeles; modelo M; fracción del cuadro guía $g = 0.8$
Salida: las tres clases más probables y los avisos que se muestran

$s \leftarrow \mathrm{round}(g \cdot \mathrm{mín}(w, h))$
$C \leftarrow$ cuadrado de lado s centrado en I ▷ la región del cuadro guía
$x \leftarrow C$ reducido a 224×224 píxeles con interpolación bilineal
$p \leftarrow M(x)$ ▷ ocho probabilidades que suman 1
$T \leftarrow$ las tres clases con mayor p
si alguna clase de T es MEL, BCC, SCC o AK y su probabilidad es al menos 0.25 **entonces**
    mostrar el aviso ámbar «Conviene revisarla con un dermatólogo»
fin si
si $\max p < 0.5$ **entonces**
    mostrar «Poco seguro: prueba con mejor luz y más cerca»
fin si
devolver T con sus probabilidades

4. Discusión

El resultado más incómodo es también el más importante: el modelo reconoce menos de la mitad de los melanomas. La mayoría de sus errores con esa clase van hacia los nevos, el desliz con peores consecuencias posibles, porque tranquiliza a quien debería consultar. La precisión del melanoma es razonable (0.761), es decir, cuando el modelo dice «melanoma» suele acertar; lo que flaquea es su disposición a decirlo. Parte de la explicación está en los pesos de la Ecuación (2): con la raíz cuadrada, el melanoma recibió un peso de 0.84, menor que el de clases mucho más raras, y el modelo terminó inclinado a no proponerlo. Un ajuste del umbral de decisión, o un peso específico para el melanoma, podría intercambiar parte de la precisión por sensibilidad, que en este contexto vale más.

Comparar nuestras cifras con las de la literatura exige cautela. Los trabajos de Esteva y de Haenssle evaluaron tareas binarias, con otros conjuntos y redes más grandes [4, 5], mientras que aquí la red elige entre ocho clases muy desbalanceadas y cabe en un archivo de 9 MB. Además, la partición entre entrenamiento y prueba no separa por lesión: HAM10000 incluye varias imágenes de una misma lesión, y si algunas quedaron de ambos lados, las cifras resultan algo optimistas. Aun con esas reservas, que una red de 2.3 millones de parámetros entrenada en 45 minutos sin tarjeta gráfica distinga nevos, carcinomas basocelulares y lesiones vasculares con recall superior a 0.79 dice bastante de lo accesible que se ha vuelto esta tecnología.

La experiencia con la Raspberry Pi dejó una lección práctica. La inferencia fue lo más sencillo, pues cada imagen tarda menos de 10 ms; lo que puso en riesgo el funcionamiento fueron la fuente y el calor. Una placa alimentada desde una laptop se apaga en cuanto se le exige todo el procesador, y una sin disipador se frena por temperatura en uno o dos minutos. Para una demostración confiable bastan la fuente

oficial de 27 W y un disipador activo, o bien limitar el modelo a dos hilos, que con 12.7 ms por imagen sigue siendo de sobra rápido.

La limitación de fondo es el tipo de imagen. El modelo aprendió con fotografías de dermatoscopio, tomadas pegadas a la piel y con luz controlada, y su acierto bajó de 24 de 40 imágenes dermatoscópicas a 7 de 22 fotos clínicas. Con una cámara común sucederá algo parecido, y lo mismo con tonos de piel poco representados en ISIC, una carencia ya documentada en otros sistemas [6]. El paso natural sería acoplar a la cámara un lente dermatoscópico económico y reentrenar con imágenes más diversas. También conviene añadir una forma de reconocer cuando la imagen no muestra una lesión, porque hoy el modelo siempre elige una de las ocho clases, incluso frente a una foto de una mano o de una taza.

5. Conclusión

Computer Vision DermaCheck demuestra que un clasificador de lesiones cutáneas puede entrenarse en una laptop y funcionar en una placa de bajo costo sin perder exactitud en el traslado: la Raspberry Pi reprodujo, imagen por imagen, las predicciones de la computadora donde se entrenó. Su desempeño, en cambio, todavía no alcanza para confiar en él frente a un melanoma, y por eso la aplicación insiste en que no diagnostica. El valor del prototipo está en otra parte, en volver tangible para un salón de clases cómo aprende una red neuronal, dónde se equivoca y por qué, y en dejar medido qué hace falta para llevarla de una computadora a un dispositivo que cabe en la mano.

Información suplementaria. El código del proyecto, los cuadernos de entrenamiento y evaluación, el modelo exportado, los registros de las pruebas en la Raspberry Pi y los créditos de las 62 imágenes del modo de prueba están en el repositorio del proyecto, disponible a solicitud de los autores.

Agradecimientos. Agradecemos a la International Skin Imaging Collaboration y a las instituciones que donaron las imágenes de ISIC 2019, a quien publicó en Hugging Face la copia del conjunto con la que trabajamos y a los fotógrafos de Wikimedia Commons que compartieron sus imágenes con licencia abierta.

Declaraciones

- **Financiamiento:** el proyecto no recibió financiamiento; el equipo cubrió el costo del hardware.
- **Conflicto de intereses:** los autores declaran no tener conflictos de intereses.
- **Aprobación ética y consentimiento para participar:** no aplica; el estudio usó solo imágenes públicas y anonimizadas.
- **Consentimiento para publicación:** no aplica.
- **Disponibilidad de datos:** el conjunto ISIC 2019 está disponible en el sitio del reto [11] y la copia utilizada, en Hugging Face [15]. Las métricas y registros de este trabajo están en el repositorio del proyecto.
- **Disponibilidad de materiales:** no aplica.

- **Disponibilidad del código:** disponible a solicitud de los autores.
- **Contribución de los autores:** todos los autores participaron en la concepción del proyecto, la preparación de los datos, el entrenamiento y la evaluación del modelo, el montaje del prototipo y la redacción del manuscrito, y aprobaron la versión final.

Apéndice A Créditos de las imágenes

Las imágenes de la Fig. 5 y de la Fig. 4 pertenecen al archivo de ISIC. La Tabla A1 indica el identificador, el autor y la licencia de cada una, según el propio archivo.

Tabla A1 Créditos de las imágenes de ISIC usadas en las figuras.

Identificador	Clase	Autor	Licencia
ISIC_0062738	AK	Hospital Clínic de Barcelona	CC BY-NC 4.0
ISIC_0029546	BCC	MILK study team	CC BY-NC 4.0
ISIC_0012323	BKL	Anónimo	CC0 1.0
ISIC_0071844	DF	Hospital Clínic de Barcelona	CC BY-NC 4.0
ISIC_0012425	MEL	Anónimo	CC0 1.0
ISIC_0014977	NV	Anónimo	CC0 1.0
ISIC_0071727	SCC	Hospital Clínic de Barcelona	CC BY-NC 4.0
ISIC_0029439	VASC	MILK study team	CC BY-NC 4.0

La Fig. 4 usa ISIC_0014977, ISIC_0029546 e ISIC_0012425.

Referencias

[1] Sung, H., Filho, A.M., Laversanne, M., Ferlay, J., Siegel, R.L., Soerjomataram, I., Jemal, A., Bray, F.: Global cancer statistics 2024: GLOBOCAN estimates of incidence and mortality worldwide for 34 cancers in 186 countries. CA: A Cancer Journal for Clinicians **76**(4), 70090 (2026) https://doi.org/10.3322/caac.70090

[2] Ervik, M., Laversanne, M., Lam, F., Colombet, M., Ferlay, J., Filho, A.M., Bray, F.: Global Cancer Observatory: Cancer Today (2024 estimates, version 1.0). Mexico: fact sheet. International Agency for Research on Cancer, Lyon, Francia. Consultado el 10 de noviembre de 2026 (2026). https://gco.iarc.who.int/media/globocan/factsheets/populations/484-mexico-fact-sheet.pdf

[3] Kittler, H., Pehamberger, H., Wolff, K., Binder, M.: Diagnostic accuracy of dermoscopy. The Lancet Oncology **3**(3), 159–165 (2002) https://doi.org/10.1016/S1470-2045(02)00679-4

[4] Esteva, A., Kuprel, B., Novoa, R.A., Ko, J., Swetter, S.M., Blau, H.M., Thrun, S.: Dermatologist-level classification of skin cancer with deep neural networks. Nature **542**(7639), 115–118 (2017) https://doi.org/10.1038/nature21056

[5] Haenssle, H.A., Fink, C., Schneiderbauer, R., Toberer, F., Buhl, T., Blum, A., Kalloo, A., Hassen, A.B.H., Thomas, L., Enk, A., Uhlmann, L., Reader study level-I and level-II Groups: Man against machine: diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists. Annals of Oncology **29**(8), 1836–1842 (2018) https://doi.org/10.1093/annonc/mdy166

[6] Daneshjou, R., Vodrahalli, K., Novoa, R.A., Jenkins, M., Liang, W., Rotemberg, V., Ko, J., Swetter, S.M., Bailey, E.E., Gevaert, O., Mukherjee, P., Phung, M., Yekrang, K., Fong, B., Sahasrabudhe, R., Allerup, J.A.C., Okata-Karigane, U., Zou, J., Chiou, A.S.: Disparities in dermatology AI performance on a diverse, curated clinical image set. Science Advances **8**(32), 6147 (2022) https://doi.org/10.1126/sciadv.abq6147

[7] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C.: MobileNetV2: Inverted residuals and linear bottlenecks. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520. IEEE, Salt Lake City, UT, USA (2018). https://doi.org/10.1109/CVPR.2018.00474

[8] International Skin Imaging Collaboration: ISIC 2019 Challenge. Consultado el 10 de noviembre de 2026 (2019). https://challenge.isic-archive.com/landing/2019/

[9] Raspberry Pi Ltd: Raspberry Pi hardware: Power supply. Raspberry Pi Documentation. Consultado el 10 de noviembre de 2026. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#power-supply

[10] Raspberry Pi Ltd: Raspberry Pi 27W USB-C Power Supply. Consultado el 10 de noviembre de 2026. https://www.raspberrypi.com/products/27w-power-supply/

[11] International Skin Imaging Collaboration: ISIC Challenge Datasets: ISIC 2019. Consultado el 10 de noviembre de 2026. https://challenge.isic-archive.com/data/

[12] Tschandl, P., Rosendahl, C., Kittler, H.: The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data **5**, 180161 (2018) https://doi.org/10.1038/sdata.2018.161

[13] Hernández-Pérez, C., Combalia, M., Podlipnik, S., Codella, N.C.F., Rotemberg, V., Halpern, A.C., Reiter, O., Carrera, C., Barreiro, A., Helba, B., Puig, S., Vilaplana, V., Malvehy, J.: BCN20000: Dermoscopic lesions in the wild. Scientific Data **11**(1), 641 (2024) https://doi.org/10.1038/s41597-024-03387-w

[14] Codella, N.C.F., Gutman, D., Celebi, M.E., Helba, B., Marchetti, M.A., Dusza, S.W., Kalloo, A., Liopyris, K., Mishra, N., Kittler, H., Halpern, A.: Skin lesion analysis toward melanoma detection: A challenge at the 2017 International Symposium on Biomedical Imaging (ISBI), hosted by the International Skin Imaging Collaboration (ISIC). In: 2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018), pp. 168–172. IEEE, Washington, DC, USA (2018).

https://doi.org/10.1109/ISBI.2018.8363547

[15] Atoany: ISIC2019. Conjunto de datos en Hugging Face. Consultado el 10 de noviembre de 2026 (2025). https://huggingface.co/datasets/Atoany/ISIC2019

[16] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861 (2017). https://doi.org/10.48550/arXiv.1704.04861 . https://arxiv.org/abs/1704.04861

[17] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L.: ImageNet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248–255. IEEE, Miami, FL, USA (2009). https://doi.org/10.1109/CVPR.2009.5206848

[18] Yosinski, J., Clune, J., Bengio, Y., Lipson, H.: How transferable are features in deep neural networks? In: Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N., Weinberger, K. (eds.) Advances in Neural Information Processing Systems, vol. 27, pp. 3320–3328. Curran Associates, Inc., Red Hook, NY, USA (2014). arXiv:1411.1792. https://arxiv.org/abs/1411.1792

[19] Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: 3rd International Conference on Learning Representations (ICLR 2015), San Diego, CA, USA (2015). arXiv:1412.6980. https://arxiv.org/abs/1412.6980

[20] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Bach, F., Blei, D. (eds.) Proceedings of the 32nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 37, pp. 448–456. PMLR, Lille, France (2015). https://proceedings.mlr.press/v37/ioffe15.html

[21] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D.G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y., Zheng, X.: TensorFlow: A system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265–283. USENIX Association, Savannah, GA, USA (2016). https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

[22] Chollet, F., et al.: Keras. Consultado el 10 de noviembre de 2026 (2015). https://keras.io

[23] He, H., Garcia, E.A.: Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering **21**(9), 1263–1284 (2009) https://doi.org/10.1109/TKDE.2008.239

[24] Google AI Edge: LiteRT: High-Performance On-Device Machine Learning Framework. Consultado el 10 de noviembre de 2026 (2026). https://developers.google.com/edge/litert

[25] Google AI Edge Team: TensorFlow Lite is now LiteRT. Google for Developers Blog. Consultado el 10 de noviembre de 2026 (2024). https://developers.googleblog.com/en/tensorflow-lite-is-now-litert/

[26] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713. IEEE, Salt Lake City, UT, USA (2018). https://doi.org/10.1109/CVPR.2018.00286

[27] Raspberry Pi Ltd: Raspberry Pi 5 Product Brief. Consultado el 10 de noviembre de 2026 (2026). https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf

[28] Google AI Edge Authors: ai-edge-litert. Paquete de Python en PyPI, versión 2.3.0. Consultado el 10 de noviembre de 2026 (2026). https://pypi.org/project/ai-edge-litert/